



Assignment 02

Subject: Programming for AI
Total Marks: 40

Post Date: 22-Oct-2025
Due Date: 14-Nov-2025

Course Instructor | Talha Shahid

Instructions

Please follow these instructions carefully to ensure your assignment is graded. Failure to adhere to these guidelines may result in penalties.

1. Deadline and Late Policy

Late Submissions: Late submissions are not permitted. Any assignment submitted after the deadline will automatically receive zero marks. Technical difficulties are not a valid excuse for late submission, so please plan to submit well before the deadline.

2. Academic Integrity: Plagiarism

- Your submission must be entirely your **own work**.
 - **Plagiarism** is strictly prohibited. This includes **copying code** from **classmates**, and **AI** (you can take guidance).
 - All submissions will be **checked** for **plagiarism**.
 - Any instance of plagiarism will result in an **automatic zero marks**.
-

3. Required Files

- You must submit a total of **05** files:
 - **One PDF Document:** A single, well-formatted PDF, renamed with your roll number (24K-####), containing the solutions for all questions. (See formatting details below).
 - **Four Code Files(.ipynb):** A separate, runnable code file for each question.
 - Please name your files clearly (e.g., **Q1.ipynb**, **Q2.ipynb**, **Q3.ipynb**, **Q4.ipynb**).
-

4. PDF Formatting Instructions

- Your single PDF document is your **main report**. It must be organized and clearly presented. For each question, you must include the following items in order:
 - Solution Code (**text format**):
 - Copy and paste your final, working code from your code file directly into the PDF.
 - **Do not use screenshots of your code.**
-



National University of Computer & Emerging Sciences, Karachi

School of Computing – BAI-3A

Fall 2025

- You must format the code properly so it is readable
-

Output Screenshot:

Immediately following the code for that question, insert a clear screenshot that shows your program running and producing the required output.

Submission Checklist:

Before you submit, check that you have:

- ☐ 1 formatted PDF file.
 - ☐ 4 separate code files (one for each question).
 - ☐ Checked that the PDF contains both the formatted code text and the output screenshot for every question.
 - ☐ Confirmed your work is original and free of plagiarism.
 - ☐ Submitted all 5 files before the deadline.
-

Submit the same on your [GitHub](#) also



Question 01:

You are a junior developer at a new sports media startup. Your first task is to build a simple, robust, command-line tool for the content team. This tool, will fetch and display a summary of all currently live cricket matches from the **cricketdata.org** API.

The senior developer has given you a set of specific requirements to ensure the tool is configurable, reliable, and provides clear output.

Task: Your script must meet the following technical specifications:

1. **Configurable Endpoint:** The API URL for "live matches" might change (e.g., to a "recent matches" endpoint later). Therefore, the script must not hardcode the URL. Instead, it must read the API URL from a local text file named `api.txt`.
2. **Robust Error Handling:** The tool will be run by non-technical users, so it must not crash. You must implement `try...except` blocks to handle:
 - a. **Network Errors:**
Gracefully handle `requests.exceptions.RequestException` (e.g., if the user has no internet connection) and print a user-friendly error.
 - b. **HTTP Errors:**
If the API returns a **non-200** status code (like **401** for a bad API key or **404**), the script must print the error `status_code` and the raw `response.text` for debugging.
3. **API Status First:** Before printing any match details, the script must first parse the JSON and print a summary of the API's own status metadata. Your manager wants to see the API usage (`hitsToday`, `hitsLimit`, etc.) at a glance. Your script must print all top-level keys except for the `'data'` key (which is too large) and the `'apikey'` key (for security).
4. **Graceful API Failures:** A **200** HTTP status doesn't always mean a successful data retrieval. The API itself might return a JSON response with `"status": "failure"`.
 - a. You must check that the JSON's `"status"` key is `"success"` and that a `"data"` key exists and is not empty.
 - b. If the status is not `"success"`, you must print the error message provided in the API's `"reason"` key.
5. **Formatted Match Display:** If all checks pass, the script must loop through the list of matches found in the `"data"` key. For each match, it must print a clearly formatted summary (using separators like `----`) showing the:
 - a. **Match Name** (name)
 - b. **Match Status** (status)
 - c. **Venue** (venue)



National University of Computer & Emerging Sciences, Karachi

School of Computing – BAI-3A

Fall 2025

d. **Date** (date)

6. **Nested Score Details:** Inside the match loop, you must check if a 'score' key exists and contains data. If it does, you must loop through the nested list of scores and print the details for each inning:
- a. **Inning** (inning)
 - b. **Runs** (r)
 - c. **Wickets** (w)
 - d. **Overs** (o)

Your final output should be a clean, readable report printed directly to the console.



Question 02:

You are an AI engineer at an agricultural tech company. The company has deployed hundreds of environmental sensors across various farms. You receive a massive, messy CSV file containing hourly soil moisture readings. Before any AI modeling can begin, you must use the high-performance capabilities of NumPy to clean and normalize this data.

The `sensor_data.csv` file has **100** columns (one for each sensor) and **720** rows (one for each hour in a **30**-day month). The data is problematic:

- Some readings are missing, represented by the string `"-999"`.
- Some sensors failed, recording impossible values (e.g., negative moisture `"-10.5"` or super-saturated `"150.0"`).
- The data is stored as strings and needs to be numerical for calculations.

Your Task:

1. **Load Data:** Read the `sensor_data.csv` file. You *cannot* use **Pandas** for this step. You must use standard Python file I/O to read the data line-by-line, split the commas, and load it into a Python list of lists.
2. **NumPy Conversion:** Convert this nested list into a 2D NumPy array.
3. **Data Cleaning (Vectorized):**
 - Use **NumPy's vectorized operations** (no for loops) to replace all occurrences of `"-999"` with `np.nan`.
 - Use **NumPy's boolean indexing** to find and replace all values less than 0 and all values greater than 100 with `np.nan`.
4. **Data Analysis (Axis Operations):**
 - Calculate the average (mean) moisture for *each sensor* (i.e., column-wise), ignoring the `np.nan` values.
 - Calculate the median moisture for *each hour* (i.e., row-wise), ignoring the `np.nan` values.
 - Identify the sensor (column index) with the *highest* number of invalid readings (the `np.nan` values you just set).



5. Normalization (Broadcasting):

- Find the minimum and maximum valid reading *across the entire dataset*.
- Use **NumPy broadcasting** to perform "**Min-Max Normalization**" on the entire array, scaling all data to be between **0** and **1**. The formula is:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

6. **Save Output:** Write the final, cleaned, and normalized NumPy array to a new CSV file named **sensor_data_normalized.csv**.

sensor_data.csv | [link](#)



Question 03:

You are a junior data scientist at a historical research institute. Your first major project is to analyze the famous **Titanic dataset** (`titanic.csv`). Your senior data scientist has told you that before any analysis, the data must be "autopsied" and meticulously cleaned. Your tool for this job is Pandas.

You must load **Titanic-Dataset.csv** into a Pandas **DataFrame** and perform a detailed cleaning and transformation process.

Your Task:

1. Load and Inspect:

- Load **Titanic-Dataset.csv** into a **DataFrame**.
- Programmatically generate a report (e.g., print to console or write to `inspect_report.txt`) that lists each column, its data type (e.g., `int64`, `object`), and the *percentage* of missing (`null`) values.

2. Handle Missing Data (Imputation):

- **'Age'**: The 'Age' column has many missing values. A simple mean is too naive. You must fill the missing 'Age' values using the *median* age of passengers in the same 'Pclass' (Passenger Class) and 'Sex' group.
- **'Embarked'**: This column has only two missing values. Fill them with the *mode* (most common value) of the 'Embarked' column.
- **'Cabin'**: This column is missing over 70% of its data. It's unusable. Drop the entire 'Cabin' column from the **DataFrame**.

3. Feature Engineering:

- Create a new column **FamilySize** by adding the 'SibSp' (siblings/spouses) and 'Parch' (parents/children) columns together.
- Create a second new column **IsAlone** that contains a 1 if **FamilySize** is 0, and a 0 otherwise.

4. **Type Conversion:** The 'Age' column might be a float after imputation. Convert it to an integer type (`int64`).

5. **Save Clean Data:** Save your cleaned, imputed, and feature-engineered **DataFrame** to a new file, `titanic_cleaned.csv`.

Titanic-Dataset.csv | [link](#)



Question 04:

After successfully cleaning the data in the previous step, your team wants to test two prevailing hypotheses about the Titanic disaster:

1. **"Women and children first."**
2. **"The wealthy had a higher survival rate."**

You will use the `titanic_cleaned.csv` you created in Question 3 and apply **Pandas** grouping and merging capabilities.

You also have a *second* file, `ticket_fares.csv`, which contains the Ticket number and the Fare paid. This data was separated for accounting reasons. To test the "wealth" hypothesis, you'll need to combine this with your main dataset.

Your Task (using `titanic_cleaned.csv`):

1. **Load Data:** Load both `titanic_cleaned.csv` and `ticket_fares.csv` into two separate DataFrames.
2. **Merge Data:** Perform a **merge** operation to combine the two DataFrames, using the 'Ticket' column as the common key. Your main DataFrame should now include the 'Fare' information.
3. **Test Hypothesis 1 (Women and Children):**
 - First, create a new categorical column **AgeGroup**. Bin the 'Age' column into logical groups: ['Child', 'Teen', 'Adult', 'Senior']. (You decide the age brackets, e.g., 0-12, 13-19, 20-59, 60+).
 - Use **groupby()** or a **pivot_table** to calculate the *mean survival rate* (**Survived** column) broken down by both 'Sex' and 'AgeGroup'.
 - Write a one-paragraph conclusion to a **report.txt** file: does the data support the "women and children first" hypothesis? Justify with your numbers.
4. **Test Hypothesis 2 (Wealth):**
 - **Method A (Class):** Calculate the mean survival rate grouped by 'Pclass' (1st, 2nd, 3rd).
 - **Method B (Fare):** The 'Fare' data is continuous. To analyze it, **bin the 'Fare' column** into 4 equal-sized quantile groups (e.g., 'Low', 'Medium', 'High', 'VeryHigh') using **pd.qcut()**. Name this new column **FareBin**.



National University of Computer & Emerging Sciences, Karachi

School of Computing – BAI-3A

Fall 2025

- Calculate the mean survival rate grouped by your new **FareBin** column.
- Append a second paragraph to **report.txt**: does the data support the "wealth" hypothesis? Compare the results from Method A and Method B.

ticket_fares.csv | [link](#)